

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении работы на тему
«Программирование и настройка СУБД»
по дисциплине «Методы проектирования баз данных»

№	Лабораторная работа	Дата сдачи
1	Представления	
2	Триггеры	
3	Права пользователей	
4	Функции и процедуры	
5	Транзакции	

Выполнил студент группы
№5130201/30101

Проверил преподаватель

Радионова П. В. 

Попов С. Г. _____

«03» 12 2025г.

Санкт-Петербург, 2025

Реферат

В ходе работы были реализованы ключевые нереляционные возможности СУБД: представления, триггеры, транзакции с заданными уровнями изоляции, управление правами пользователей, а также хранимые процедуры и функции. Объем реализации включает пять этапов, в которых последовательно разработаны представление для агрегации данных, триггеры для учёта продаж, систему разграничения доступа, сценарии транзакций с демонстрацией аномалий, а также бизнес-логику в виде процедур и функций. Для реализации использовалась СУБД MySQL. Полученные результаты могут быть применены для автоматизации бизнес-процессов ветеринарной клиники, таких как автоматический учёт продаж и управление ассортиментом. Реализованная система также обеспечивает целостность данных, контролируемый доступ и демонстрирует работу с транзакционными аномалиями, что полезно для обучения и внедрения в реальных базах данных.

Содержание

Введение	4
1 Постановка задачи	5
2 Представления	6
2.1 Создание представления	6
2.2 Выполнение запросов	7
2.3 Проверка доступа к представлению	8
3 Триггеры	10
3.1 Проектирование и создание таблицы	10
3.2 Создание триггеров	12
3.3 Проверка корректности работы триггеров	14
4 Права пользователей	18
4.1 Создание пользователей	18
4.2 Изменение прав пользователей	18
4.3 Проверка прав доступа	19
5 Функции и процедуры	21
5.1 Создание функции	21
5.2 Проверка работы функции	22
5.3 Создание процедуры	23
5.4 Проверка работы процедуры	24
6 Транзакции	27
6.1 Сценарий выполнения транзакций	27
6.2 Результаты выполнения транзакций	28
Заключение	30
Список использованных источников	31
Приложение А. «Исходный текст функции»	32
Приложение Б. «Исходный текст процедуры»	33

Введение

Работа современных ветеринарных клиник строится на обработке больших и постоянно растущих объемов информации. Ежедневно необходимо учитывать десятки пациентов — домашних питомцев с уникальными историями болезней, прививок и особенностей. Параллельно ведется запись на приёмы, оформление медицинских назначений и рекомендаций, а также ведение аптечного склада и продажа специализированных товаров: кормов, аксессуаров, средств гигиены и лекарственных препаратов. Каждый владелец животного представляет собой отдельную учетную единицу с контактными данными, историей обращений и финансовых расчетов. Такое количество взаимосвязанных данных создает потребность в четкой и прозрачной системе учета, где информация о питомце, его здоровье, посещениях и покупках была бы целостной, актуальной и легко доступной для персонала. Именно базы данных обеспечивают хранение, структурирование и доступ ко всей необходимой информации для эффективного управления медицинскими и коммерческими процессами.

1 Постановка задачи

1. Создать представление и выполнить запросы к нему.
2. Создать таблицу учета продаж и создать триггеры к ней.
3. Создать двух новых пользователей, изменить их права и выполнить запросы от их лица.
4. Написать хранимые функцию и процедуру.
5. Написать сценарий выполняемых транзакций на выбранном уровне изоляции и продемонстрировать наличие или отсутствие аномалий на этом уровне.

2 Представления

На данном этапе работы требуется создать представление на основе уже спроектированной базы данных и написать запросы, обращающиеся к этому представлению.

Постановка задачи

1. Создание представления на основе спроектированной ранее схемы базы данных.
2. Написать и выполнить простой запрос к созданному представлению.
3. Написать и выполнить сложный запрос к созданному представлению.
4. Проверить право доступа на изменение представления.

2.1 Создание представления

На основе спроектированной ранее схемы базы данных был сформулирован следующий запрос: посчитать количество покупок и общую сумму всех покупок для каждого владельца. На основе этого запроса было создано представление.

Код создания представления

Код создания представления приведен в листинге 1.

```
1 CREATE VIEW v AS
2     SELECT
3         o.owner_id AS id,
4         o.first_name AS owner_name ,
5         o.second_name AS owner_surname ,
6         COUNT(p.purchase_id) AS number_of_purchase ,
7         SUM(pr.price) AS total_spent
8     FROM
9         owners o
10        JOIN
11        purchase p ON o.owner_id = p.owner_id
12        JOIN
13        product pr ON p.product_id = pr.product_id
14    GROUP BY o.owner_id , owner_name
15    ORDER BY total_spent DESC
```

Листинг 1: Код создания представления

2.2 Выполнение запросов

Формулировка простого запроса

Найти всех владельцев, общая сумма покупок которых превышает 150000.

Код простого запроса

Код простого запроса приведен в листинге 2.

```
1 SELECT
2     owner_name , total_spent
3 FROM
4     v
5 WHERE
6     total_spent > 150000
7 ORDER BY total_spent DESC;
```

Листинг 2: Код простого запроса

Результат выполнения простого запроса

В результирующей таблице было выведено 98 строк. Фрагмент результирующей таблицы приведен на рис. 1.

owner_name	total_spent
Никанор	217553.76
Анжелика	211882.40
Любомир	211129.12
Василиса	210059.33
Устин	209105.61
Остромир	208661.37
Наум	208010.35
Аполлон	205716.50
Анисим	204936.68

Рис. 1: Результат выполнения простого запроса к представлению в виде фрагмента таблицы

Формулировка сложного запроса

Найти количество приемов для первых 10 самых щедрых владельцев.

Код сложного запроса

Код сложного запроса приведен в листинге 3.

```
1 SELECT
2     o.owner_id AS id,
3     o.first_name AS owner_name,
4     o.second_name AS owner_surname,
5     COUNT(a.appointment_id) AS amount_of_apps
6 FROM
7     v
8     JOIN
9     owners o ON v.id = o.owner_id
10    LEFT JOIN
11    appointment a ON o.owner_id = a.owner_id
12 GROUP BY v.id, v.owner_name, v.owner_surname
13 ORDER BY amount_of_apps DESC
14 LIMIT 10;
```

Листинг 3: Код сложного запроса

Результат выполнения сложного запроса

В результирующей таблице было выведено 10 строк. Фрагмент результирующей таблицы приведен на рис. 2.

id	owner_name	owner_surname	amount_of_apps
39	Ефрем	Давыдов	1091
8	Лукьян	Овчинников	1091
22	Капитон	Вишняков	1087
84	Наина	Никонова	1073
170	Кира	Сорокина	1067
36	Виталий	Рожков	1059
7	Владилен	Кузнецов	1059
56	Федосий	Туров	1054
173	Евстафий	Фокин	1045
43	Орест	Веселов	1042

Рис. 2: Результат выполнения сложного запроса к представлению в виде фрагмента таблицы

2.3 Проверка доступа к представлению

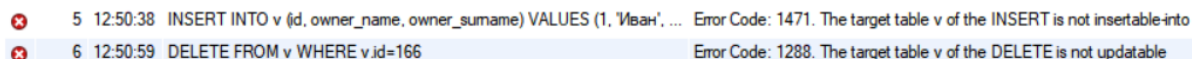
В созданном представлении присутствуют агрегатные функции `COUNT()` и `SUM()`, а также группировка `GROUP BY()`. Это делает представление не обновляемым.

Для демонстрации того, что созданное представление доступно только для чтения, был произведен запрос со вставкой нового значения в таблицу представления. Код запроса приведен в листинге 4.

```
1 INSERT INTO v (id, owner_name, owner_surname) VALUES (1, 'Иван',  
    'Иванов');
```

Листинг 4: Вставка нового значения в представление

Результат попыток изменения таблицы представления представлен на рис. 3.



```
5 12:50:38 INSERT INTO v (id, owner_name, owner_surname) VALUES (1, 'Иван', ... Error Code: 1471. The target table v of the INSERT is not insertable-into  
6 12:50:59 DELETE FROM v WHERE v.id=166 Error Code: 1288. The target table v of the DELETE is not updatable
```

Рис. 3: Проверка доступа

3 Триггеры

На этом этапе требуется добавить в уже спроектированную базу данных новую таблицу с учетом продаж товаров ветеринарной клиники, а также создать триггеры, которые бы моделировали подсчет проданных товаров каждого типа.

Постановка задач

1. Составить таблицу с атрибутами новой таблицы учета продаж товаров.
2. Составить схему базы данных с таблицей учета продаж товаров на основе ранее составленной схемы базы данных.
3. Создать таблицу учета продаж товаров в ранее созданной базе данных.
4. Создать триггеры для таблицы с товарами, которые учитывали бы появление новых товаров и их удаление в таблице продаж.
5. Создать триггеры для таблицы с покупками, которые учитывали бы количество проданных товаров в таблице продаж.

3.1 Проектирование и создание таблицы

Таблица с описанием атрибутов

В таблице 1 представлено описание атрибутов таблицы продаж. Ограничение для атрибута **product_name** аналогично ограничению этого атрибута в таблице-словаре. Для атрибута **sells** выбрано значение по умолчанию 0.

Таблица 1: Инвентаризация товаров - inventory_products

Инвентаризация товаров - inventory_products						
№	Тип ключа	Название RU	Название EN	Тип атрибута	NULL/NOT NULL	Ссылка
1	PK	Товар_id	product_id	INT	NOT NULL	-
2	-	Товар_название	product_name	VARCHAR(50)	NOT NULL	-
3	-	Продажи	sells	INT	DEFAULT 0	-

Графическое описание схемы базы данных

Графическое описание схемы базы данных на английском языке с новой таблицей, триггерами и направлением их действия представлено на рис. 4.

Графическое описание схемы базы данных на английском языке

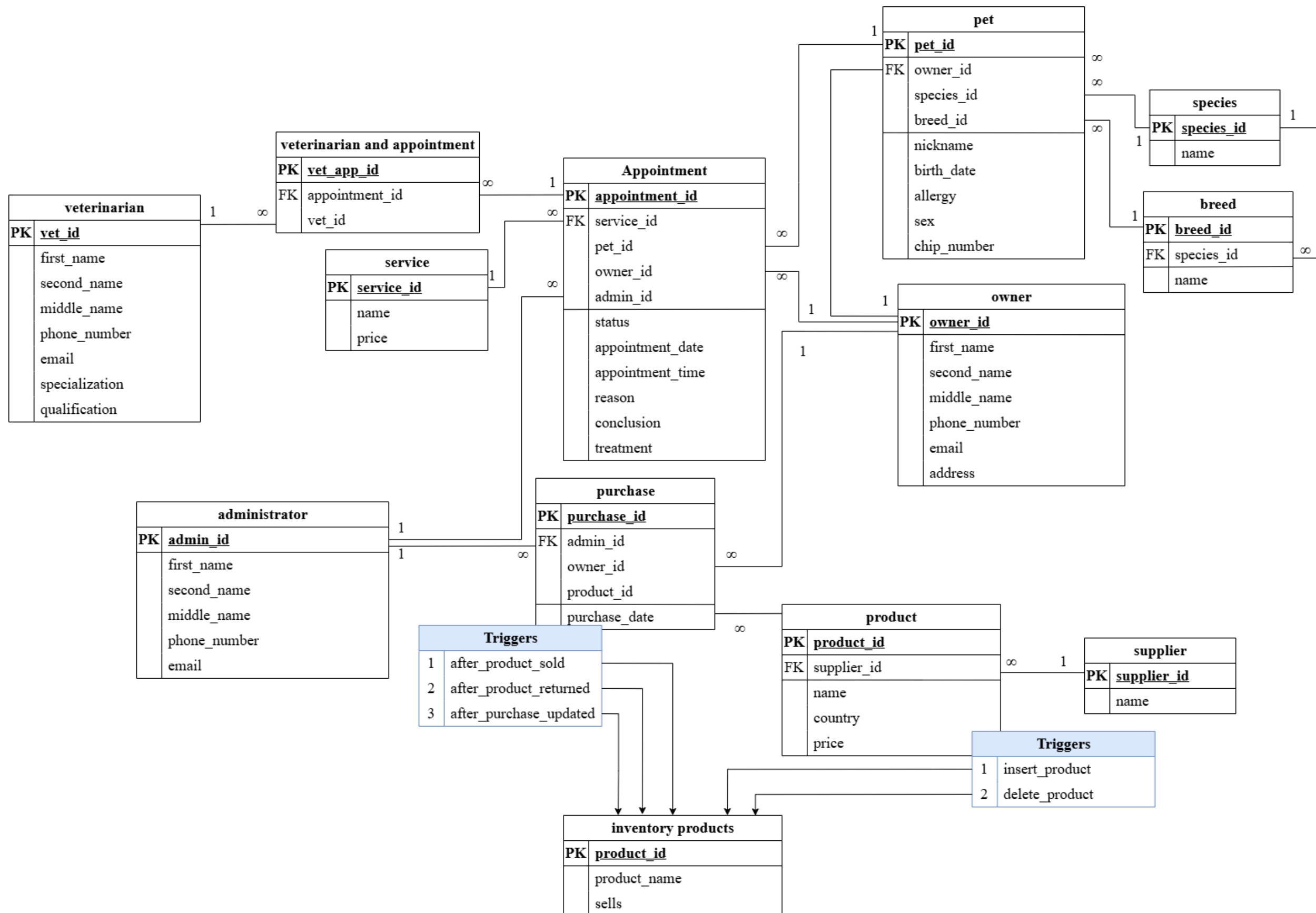


Рис. 4. Схема базы данных на английском языке

Код создания таблицы

В листинге 5 приведен код создания таблицы с продажами и её заполнения. Названия товаров и их ID были вставлены из уже имеющейся таблицы `product`, а данные для атрибута `sales` были определены запросом к таблице `purchase`, считающим, сколько покупок было совершено для каждого товара.

```
1 CREATE TABLE inventory_products (  
2     product_id INT PRIMARY KEY,  
3     product_name VARCHAR(50) NOT NULL,  
4     sales INT DEFAULT 0  
5 );  
6  
7 INSERT INTO inventory_products(product_id, product_name, sales)  
8 SELECT  
9     p.product_id,  
10    p.name as product_name,  
11    COALESCE(purchase_count, 0) as sales  
12 FROM product p  
13 LEFT JOIN (  
14     SELECT product_id, COUNT(*) as purchase_count  
15     FROM purchase  
16     GROUP BY product_id  
17 ) pc ON p.product_id = pc.product_id;
```

Листинг 5: Код создания и заполнения таблицы

3.2 Создание триггеров

Для обработки добавления нового товара в таблицу `product` был написан триггер `insert_product`, который добавляет новый товар в таблицу `inventory_products` и присваивает атрибуту `sales` значение 0. Код триггера приведен в листинге 6.

```
1 DELIMITER //  
2  
3 CREATE TRIGGER insert_product  
4 AFTER INSERT ON product  
5 FOR EACH ROW  
6 BEGIN  
7     INSERT INTO inventory_products (product_id, product_name,  
8         sales)  
9     VALUES (NEW.product_id, NEW.name, 0);  
10 END//
```

Листинг 6: Код создания триггера

Для обработки удаления товара из таблицы `product` был написан триггер `delete_product`, который также удаляет этот товар из `inventory_products`. Код триггера приведен в листинге 7.

```

1 CREATE TRIGGER delete_product
2 AFTER DELETE ON product
3 FOR EACH ROW
4 BEGIN
5     DELETE FROM inventory_products
6     WHERE product_id = OLD.product_id;
7 END//

```

Листинг 7: Код создания триггера

Для обработки новой покупки был написан триггер `after_product_sold`, который увеличивает значение атрибута `sales` в `inventory_products` для проданного товара на 1. Код триггера приведен в листинге 8.

```

1 CREATE TRIGGER after_product_sold
2 AFTER INSERT ON purchase
3 FOR EACH ROW
4 BEGIN
5     UPDATE inventory_products
6     SET sales = sales + 1
7     WHERE product_id = NEW.product_id;
8 END//

```

Листинг 8: Код создания триггера

Для обработки возврата товара был написан триггер `after_product_returned`, который уменьшает значение атрибута `sales` в таблице `inventory_products` для возвращенного товара на 1. Код создания триггера приведен в листинге 9.

```

1 CREATE TRIGGER after_product_returned
2 AFTER DELETE ON purchase
3 FOR EACH ROW
4 BEGIN
5     UPDATE inventory_products
6     SET sales = sales - 1
7     WHERE product_id = OLD.product_id;
8 END//

```

Листинг 9: Код создания триггера

Для обработки изменения данных о проданном товаре, то есть в случае изменения ID проданного товара в таблице `purchase`, был создан триггер, который увеличивает значение атрибута `sales` для нового товара и уменьшает значение этого же атрибута для старого товара на 1. Код создания триггера приведен в листинге 10.

```

1 CREATE TRIGGER after_purchase_updated
2 AFTER UPDATE ON purchase
3 FOR EACH ROW
4 BEGIN
5     IF OLD.product_id != NEW.product_id THEN

```

```

6      UPDATE inventory_products
7      SET sales = sales - 1
8      WHERE product_id = OLD.product_id;
9
10     UPDATE inventory_products
11     SET sales = sales + 1
12     WHERE product_id = NEW.product_id;
13 END IF;
14 END //
15
16 DELIMITER ;

```

Листинг 10: Код создания триггера

3.3 Проверка корректности работы триггеров

Для проверки корректности работы триггеров были сформированы тестовые примеры, которые покрывают все варианты изменения данных исходных таблиц. Все тестовые примеры приведены в таблице 2.

Таблица 2: Тестовые примеры для проверки триггеров

№	Описание проверяемого состояния
1	Добавление нового товара в таблицу <code>product</code> . В <code>inventory_products</code> автоматически создаётся запись с нулевыми продажами.
2	Удаление существующего товара из таблицы <code>product</code> . Соответствующая запись удаляется из <code>inventory_products</code> .
3	Добавление покупки в <code>purchase</code> . Поле <code>sales</code> для данного товара увеличивается на 1.
4	Удаление записи о покупке в <code>purchase</code> . Поле <code>sales</code> для данного товара уменьшается на 1.
5	Обновление записи в <code>purchase</code> , при котором <code>product_id</code> изменяется. У старого товара <code>sales</code> уменьшилось на 1, а у нового увеличилось на 1.
6	Обновление записи в <code>purchase</code> , при котором <code>product_id</code> остаётся тем же (изменяются другие поля). В <code>inventory_products</code> не происходит изменений.

После добавления нового товара в таблицу `product`, проверим его наличие в таблице `inventory_products`. На рис. 5 можно увидеть, что новая в таблице `product` запись также добавлена в таблицу `inventory_products`. Таким образом, триггер `insert_product` работает корректно.

Для проверки корректности работы триггера `delete_product`, удалим только что добавленный товар. Сначала произведем запрос с `DELETE` к таблице `product`, а затем проверим наличие этого товара в таблице `inventory_pro-`

```
mysql> INSERT INTO product (product_id, name, country, price, supplier_id)
-> VALUES (999, 'Dog food', 'UK', 1000.00, 1);
Query OK, 1 row affected (0.13 sec)

mysql> SELECT * FROM inventory_products WHERE product_id = 999;
+-----+-----+-----+
| product_id | product_name | sales |
+-----+-----+-----+
|          999 | Dog food      |      0 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

Рис. 5: Проверка работы триггера insert_product

ducts. На рис. 6 можно увидеть, что после удаления товара из таблицы product, этот товар также был удален из таблицы inventory_products.

```
mysql> DELETE FROM product WHERE product_id = 999;
Query OK, 1 row affected (0.03 sec)

mysql> SELECT * FROM inventory_products WHERE product_id = 999;
Empty set (0.01 sec)
```

Рис. 6: Проверка работы триггера delete_product

Для проверки работы триггера after_product_sold для начала посмотрим, сколько продаж у товара с product_id = 1. Произведя простой запрос с SELECT увидим, что этот товар был продан 7 раз. Затем вставим новую запись в таблицу purchase с product_id = 1. После вставки проверим еще раз количество продаж. На рис. 7 можно увидеть, что после вставки новой записи в таблицу с покупками, количество продаж у товара увеличилось на 1.

```
mysql> SELECT product_id, product_name, sales
-> FROM inventory_products
-> WHERE product_id = 1;
+-----+-----+-----+
| product_id | product_name | sales |
+-----+-----+-----+
|          1 | Расческа     |      7 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> INSERT INTO purchase (purchase_date, product_id, owner_id, admin_id)
-> VALUES (CURDATE(), 1, 1, 1);
Query OK, 1 row affected (0.02 sec)

mysql> SELECT product_id, product_name, sales
-> FROM inventory_products
-> WHERE product_id = 1;
+-----+-----+-----+
| product_id | product_name | sales |
+-----+-----+-----+
|          1 | Расческа     |      8 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

Рис. 7: Проверка работы триггера after_product_sold

Для проверки триггера `after_product_returned` выберем первую запись в таблице `purchase`. Для демонстрации возьмем товар с `ID = 481`. Произведя запрос с `SELECT`, увидим, что этот товар был продан 13 раз. Удалим первую запись из таблицы покупок, а затем снова посмотрим на товар с `ID = 481` в таблице `inventory_products`. На рис. 8 можно увидеть, что после возврата товара количество продаж этого товара уменьшилось на 1.

```
mysql> SELECT * FROM purchase LIMIT 1;
+-----+-----+-----+-----+-----+
| purchase_id | purchase_date | product_id | owner_id | admin_id |
+-----+-----+-----+-----+-----+
| 1 | 2025-04-29 | 481 | 1 | 17 |
+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)

mysql> SELECT product_id, product_name, sales
-> FROM inventory_products
-> WHERE product_id = 481;
+-----+-----+-----+
| product_id | product_name | sales |
+-----+-----+-----+
| 481 | Лекарства витамины песенка | 13 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> DELETE FROM purchase WHERE purchase_id = 1;
Query OK, 1 row affected (0.04 sec)

mysql> SELECT product_id, product_name, sales
-> FROM inventory_products
-> WHERE product_id = 481;
+-----+-----+-----+
| product_id | product_name | sales |
+-----+-----+-----+
| 481 | Лекарства витамины песенка | 12 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

Рис. 8: Проверка работы триггера `after_product_returned`

Для проверки работы триггера `after_purchase_updated` просмотрим продажи товаров с `ID 2` и `3`, заменим `ID` товара с `2` на `3` в какой-нибудь записи в `purchase` и просмотрим количество продаж еще раз. На рис. 9 можно увидеть, как у одного товара продажи уменьшились на 1, а у другого увеличились на 1.

```
mysql> SELECT product_id, sales FROM inventory_products WHERE product_id IN (2, 3);
+-----+-----+
| product_id | sales |
+-----+-----+
| 2 | 17 |
| 3 | 1 |
+-----+-----+
2 rows in set (0.00 sec)

mysql> UPDATE purchase SET product_id = 3 WHERE purchase_id = 223;
Query OK, 1 row affected (0.03 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> SELECT product_id, sales FROM inventory_products WHERE product_id IN (2, 3);
+-----+-----+
| product_id | sales |
+-----+-----+
| 2 | 16 |
| 3 | 2 |
+-----+-----+
2 rows in set (0.00 sec)
```

Рис. 9: Проверка работы триггера `after_purchase_updated`

Если не изменять ID товара, а изменять какие-то другие его атрибуты, то количество продаж в таблице `inventory_products` изменяться не будет. Это можно увидеть на рис. 10.

```
mysql> SELECT product_id, sales FROM inventory_products WHERE product_id = 592;
+-----+-----+
| product_id | sales |
+-----+-----+
|          592 |      19 |
+-----+-----+
1 row in set (0.00 sec)

mysql> UPDATE purchase SET admin_id = 18 WHERE purchase_id = 3;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT product_id, sales FROM inventory_products WHERE product_id = 592;
+-----+-----+
| product_id | sales |
+-----+-----+
|          592 |      19 |
+-----+-----+
1 row in set (0.00 sec)
```

Рис. 10: Проверка работы триггера `after__purchase__updated`

4 Права пользователей

Требуется создать двух новых пользователей и изменить их права так, чтобы один из пользователей мог только просматривать таблицу `inventory_products`, а другой мог управлять всеми таблицами, на которых висят созданные ранее триггеры, то есть `product` и `purchase`.

Постановка задач

1. Создать 2 новых пользователей.
2. Изменить права созданных пользователей так, чтобы один из них мог только просматривать таблицу `inventory_products`, а другой мог управлять всеми таблицами, на которых висят созданные триггеры.
3. Выполнить запросы от разных пользователей и продемонстрировать ограничения, обусловленные их правами доступа.

4.1 Создание пользователей

Было создано 2 новых пользователя с именами `user1` и `user2`. `user1` был наделен правами администратора, а `user2` получил права на вставку, удаление и изменение данных в базе данных. Код создания пользователей приведен в листинге 11.

```
1 CREATE USER 'user1'@'localhost' IDENTIFIED BY '12345';
2 GRANT ALL PRIVILEGES ON vetClinic.* TO 'user1'@'localhost';
3
4 CREATE USER 'user2'@'localhost' IDENTIFIED BY '12345';
5 GRANT SELECT, INSERT, UPDATE ON vetClinic.* TO 'user2'@'localhost';
```

Листинг 11: Создание пользователей

4.2 Изменение прав пользователей

Для созданных ранее пользователей были изменены права: `user1` теперь имеет доступ только к просмотру таблицы `inventory_products`, а `user2` обладает всеми правами на таблицы `product` и `purchase`. Код назначения прав приведен в листинге 12.

```
1 REVOKE ALL PRIVILEGES ON vetClinic.* FROM 'user1'@'localhost';
2 REVOKE ALL PRIVILEGES ON vetClinic.* FROM 'user2'@'localhost';
3
4 GRANT SELECT ON vetClinic.inventory_products TO 'user1'@'localhost';
5
```

```

6 GRANT ALL PRIVILEGES ON vetClinic.product TO 'user2'@'localhost';
7 GRANT ALL PRIVILEGES ON vetClinic.purchase TO 'user2'@'localhost'
8 ;
9 FLUSH PRIVILEGES;

```

Листинг 12: Изменение прав пользователей

4.3 Проверка прав доступа

На рис. 11 приведены таблицы с правами доступа пользователей **user1** и **user2**.

```

mysql> SHOW GRANTS FOR 'user1'@'localhost';
+-----+
| Grants for user1@localhost |
+-----+
| GRANT USAGE ON *.* TO 'user1'@'localhost' |
| GRANT SELECT ON `vetclinic`.`inventory_products` TO 'user1'@'localhost' |
+-----+
2 rows in set (0.01 sec)

mysql> SHOW GRANTS FOR 'user2'@'localhost';
+-----+
| Grants for user2@localhost |
+-----+
| GRANT USAGE ON *.* TO 'user2'@'localhost' |
| GRANT ALL PRIVILEGES ON `vetclinic`.`product` TO 'user2'@'localhost' |
| GRANT ALL PRIVILEGES ON `vetclinic`.`purchase` TO 'user2'@'localhost' |
+-----+
3 rows in set (0.00 sec)

```

Рис. 11: Таблицы прав доступа пользователей

Далее был выполнен один и тот же перечень запросов от обоих пользователей. В таблице 3 приведены результаты запросов для каждого из пользователей.

Таблица 3: Права доступа пользователей к таблицам базы данных

№	user1	user2
1	SELECT * FROM inventory_products;	
	Таблица inventory_products	SELECT command denied to user 'user2'@'localhost' for table 'inventory_products'
2	SELECT * FROM product;	
	SELECT command denied to user 'user1'@'localhost' for table 'product'	Таблица product

№	user1	user2
3	SELECT * FROM purchase;	
	SELECT command denied to user 'user1'@'localhost' for table 'purchase'	Таблица purchase
4	INSERT INTO product (name, country, price, supplier_id) VALUES ('Корм холистик', 'Норвегия', 3600, 19);	
	INSERT command denied to user 'user1'@'localhost' for table 'product'	Выполненный запрос
5	INSERT INTO purchase(purchase_date, product_id, owner_id, admin_id) VALUES ('1990-10-11', 1, 2, 3);	
	INSERT command denied to user 'user1'@'localhost' for table 'purchase'	Выполненный запрос
6	UPDATE product SET price = price * 1.1 WHERE product_id = 12;	
	UPDATE command denied to user 'user1'@'localhost' for table 'product'	Выполненный запрос
7	UPDATE purchase SET product_id = 3 WHERE purchase_id = 156;	
	UPDATE command denied to user 'user1'@'localhost' for table 'purchase'	Выполненный запрос
8	DELETE FROM product WHERE product_id=700;	
	DELETE command denied to user 'user1'@'localhost' for table 'product'	Выполненный запрос
9	DELETE FROM purchase WHERE product_id=1 AND admin_id=1;	
	DELETE command denied to user 'user1'@'localhost' for table 'purchase'	Выполненный запрос
10	SHOW TRIGGERS;	
	Empty set.	Таблица триггеров таблиц product и purchase.

5 Функции и процедуры

Требуется написать SQL-функцию, которая собирала бы одно поле фамилии с инициалами из 3-х отдельных полей с фамилией, именем и отчеством. Также нужно написать SQL процедуру, которая принимает название поставщика, название товара, страну производства товара и его цену, обрабатывает, есть ли переданный поставщик в таблице `supplier` и добавляет его в таблицу, если его еще нет, а также добавляет новый товар для данного поставщика, если такого товара еще нет.

Постановка задачи

1. Написать функцию, которая принимала бы поля, содержащие фамилию, имя и отчество, производила бы конкатенацию полей и возвращала одно поле с фамилией и инициалами.
2. Написать процедуру, которая принимала бы название поставщика, название товара, страну производства товара и его цену и работала бы по следующей логике:
 - (a) Если переданного поставщика нет в таблице поставщиков, то добавить его в таблицу и вернуть его ID.
 - (b) Если переданный поставщик уже есть в таблице поставщиков, то вернуть его ID.
 - (c) Если переданный товар уже есть в таблице товаров у данного поставщика, то вернуть его ID.
 - (d) Если переданного товара еще нет у данного поставщика, то добавить этот товар в таблицу товаров и вернуть его ID.
3. Продемонстрировать работу функции.
4. Продемонстрировать работу процедуры.

5.1 Создание функции

Функция `get_surname_with_initials` принимает 3 поля типа `VARCHAR`, которые соответствуют фамилии, имени и отчеству и возвращает одно поле с фамилией и инициалами, разделенными точкой. Если у записи поле отчества равняется `NULL`, то этот инициал пропускается. Код функции приведен в листинге 17.

5.2 Проверка работы функции

Для демонстрации работы функции были сформированы тестовые примеры, покрывающие все варианты реализации функции. Примеры приведены в таблице 4.

Таблица 4: Тестовые примеры для проверки функции

№	Описание проверяемого состояния
1	У владельца заполнены все три поля: <code>second_name</code> , <code>first_name</code> , <code>middle_name</code> .
2	У владельца отсутствует значение атрибута <code>second_name</code> .

Для проверки работы функции был выполнен запрос, который применяет функцию к атрибутам `second_name`, `first_name` и `middle_name` таблицы `owners`. Код запроса приведен в листинге 13.

```
1 SELECT
2     owner_id ,
3     second_name ,
4     first_name ,
5     middle_name ,
6     get_surname_with_initials(second_name , first_name ,
7                               middle_name) as initials
FROM owners ;
```

Листинг 13: Запрос с применением функции

Фрагмент таблицы результатов запроса, демонстрирующий работу функции, приведен на рис. 12.

owner_id	second_name	first_name	middle_name	initials
1	Лобанов	Герасим	NULL	Лобанов Г.
2	Фадеева	Иванна	Макаровна	Фадеева И.М.
3	Николаев	Милен	Феоктистович	Николаев М.Ф.
4	Прохорова	Оксана	Ждановна	Прохорова О.Ж.
5	Кулаков	Максимильян	Чеславович	Кулаков М.Ч.
6	Блинов	Архип	Зиновьевич	Блинов А.З.
7	Кузнецов	Владилен	Викторович	Кузнецов В.В.
8	Овчинников	Лукьян	Денисович	Овчинников Л.Д.
9	Авдеева	Пелагея	Ждановна	Авдеева П.Ж.
10	Журавлев	Серафим	Артёмович	Журавлев С.А.
11	Бобылева	Евфросиния	Владиславовна	Бобылева Е.В.
12	Лобанова	Жанна	Степановна	Лобанова Ж.С.
13	Харитонов	Платон	Фокич	Харитонов П.Ф.

Рис. 12: Фрагмент таблицы выполнения запроса с функцией

5.3 Создание процедуры

Перед написанием кода процедуры была составлена блок-схема, описывающая алгоритм работы процедуры. Блок-схема приведена на рис. 13.

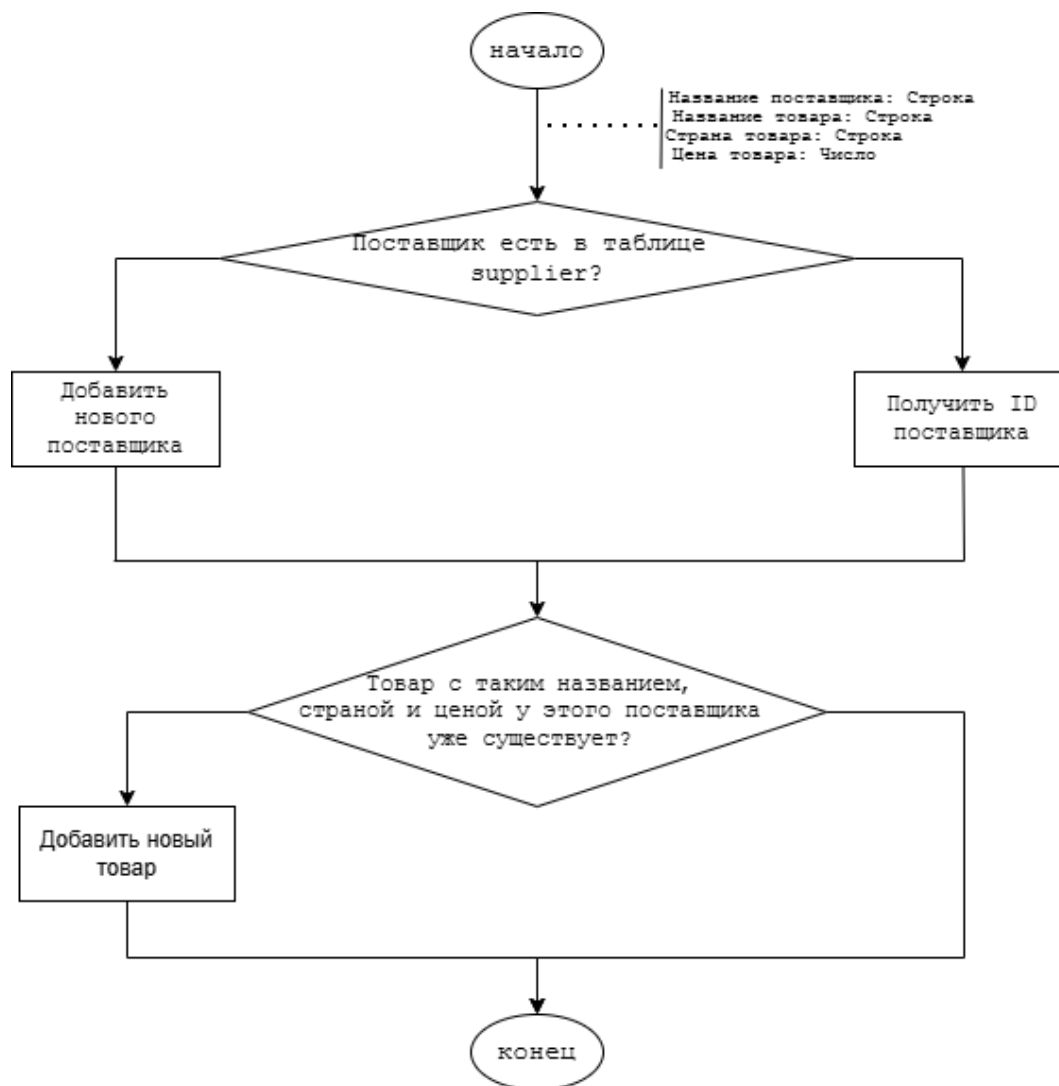


Рис. 13: Блок-схема работы процедуры

По алгоритму, продемонстрированном на блок-схеме, был написан код создания процедуры `add_product_with_supplier`. Эта процедура выполняет операцию добавления товара в систему с автоматической проверкой и созданием связанного поставщика. Она принимает четыре входных параметра: название поставщика, название товара, страну происхождения товара и его цену.

Первым этапом процедура проверяет наличие поставщика с указанным названием в таблице `supplier`. Если поставщик не найден, процедура автоматически создает новую запись в таблице поставщиков и сохраняет полученный идентификатор в переменную `supplier_id_new`. Если поставщик уже существует, процедура просто запоминает его идентификатор.

На втором этапе процедура проверяет, существует ли уже товар с указанными характеристиками у данного поставщика. Проверка осуществляется по четырем параметрам, которые передаются функции. Если товар с указанными параметрами не найден, процедура выполняет вставку новой записи в таблицу `product`, связывая товар с поставщиком через сохраненный идентификатор. В случае, когда товар уже существует в таблице, процедура завершает работу без внесения изменений.

Код создания процедуры приведен в листинге 18.

5.4 Проверка работы процедуры

Для демонстрации работы процедуры были сформированы тестовые примеры, покрывающие все варианты реализации функции. Примеры приведены в таблице 5.

Таблица 5: Тестовые примеры для проверки процедуры

№	Описание проверяемого состояния
1	Обращение к уже существующему поставщику с уже существующим у него товаром.
2	Обращение к уже существующему поставщику по несуществующему товару.
3	Обращение к не существующему поставщику.

Для начала был выполнен запрос с обращением к уже существующему поставщику с уже существующим у него товаром. Код вызова функции приведен в листинге 14.

```
1 CALL add_product_with_supplier('Богданов ИНК', 'Игрушки мячи тута', 'Италия', 1100.49);
```

Листинг 14: Запрос с применением функции

Результат вызова процедуры приведен на рис. 14. Новых записей ни в одну из таблиц добавлено не было.

```
mysql> CALL add_product_with_supplier('Богданов ИНК', 'Игрушки мячи тута', 'Италия', 1100.49);
+-----+
| message |
+-----+
| Поставщик уже существует. ID: 30 |
+-----+
1 row in set (0.00 sec)

+-----+
| message |
+-----+
| Товар "Игрушки мячи тута" от поставщика "Богданов ИНК" уже существует |
+-----+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.02 sec)
```

Рис. 14: Результат вызова процедуры при совпадении ввода с существующей записью

Далее был выполнен вызов процедуры с обращением к уже существующему поставщику, но по несуществующему товару. Код вызова процедуры приведен в листинге 15.

```
1 CALL add_product_with_supplier('Богданов ИНК', 'Наполнитель для лотка', 'Россия', 800.50);
```

Листинг 15: Запрос с применением функции

Результат вызова процедуры приведен на рис. 15. В таблицу товаров была добавлена новая запись.

```
mysql> CALL add_product_with_supplier('Богданов ИНК', 'Наполнитель для лотка', 'Россия', 800.50);
+-----+
| message |
+-----+
| Поставщик уже существует. ID: 30 |
+-----+
1 row in set (0.00 sec)

+-----+
| message |
+-----+
| Добавлен новый товар: Наполнитель для лотка от поставщика Богданов ИНК |
+-----+
1 row in set (0.01 sec)
```

Рис. 15: Результат вызова процедуры при совпадении только поставщика с существующей записью

На рис. 16 приведен фрагмент таблицы, демонстрирующий новую запись товара с `supplier_id=30` в таблице `product`. Таблица `supplier` остается неизменной.

product_id	name	country	price	supplier_id
95	Игрушки канаты фонарик	Франция	548.19	29
466	Гигиена пеленки июнь	Италия	4732.77	29
686	Аксессуары переноски порт	Великобритания	2932.61	29
429	Игрушки мячи тута	Италия	1100.49	30
490	Корм премиум песня	Испания	4103.61	30
711	Наполнитель для лотка	Россия	800.50	30
122	Игрушки пищалки спалить	Китай	4853.61	31

Рис. 16: Таблица `product` с новой записью

В заключение был выполнен вызов процедуры с параметрами, не совпадающими с существующими записями в таблице. Код вызова процедуры приведен в листинге 16.

```
1 CALL add_product_with_supplier('ЗооМир', 'Корм для кошек Premium', 'Норвегия', 1200.00);
```

Листинг 16: Запрос с применением функции

Результат вызова процедуры приведен на рис. 17. В таблицы `supplier` и `product` были добавлены новые записи.

```
mysql> CALL add_product_with_supplier('ЗооМир', 'Корм для кошек Premium', 'Норвегия', 1200.00);
+-----+-----+
| message |
+-----+-----+
| Добавлен новый поставщик: ЗооМир с ID: 152 |
+-----+-----+
1 row in set (0.03 sec)

+-----+-----+
| message |
+-----+-----+
| Добавлен новый товар: Корм для кошек Premium от поставщика ЗооМир |
+-----+-----+
1 row in set (0.04 sec)
```

Рис. 17: Результат вызова процедуры при полном не совпадении с существующими записями

На рис. 18 приведены таблицы `supplier` и `product` с новыми записями, добавленными в результате вызова процедуры.

supplier_id	name
138	ОАО «Пестова, Дмитриева и Комиссаров»
139	Макоимова и партнеры
140	Сандоз (Sandoz)
141	Дмитриев Групп
142	ЗАО «Новикова»
143	РАО «Волкова»
144	ООО «Фадеев»
145	Владимиров и партнеры
146	ИП «Ситников, Сафонова и Павлов»
147	РАО «Одинцова-Блинова»
148	ЗАО «Емельянова-Архипова»
149	АО «Зыков-Миронова»
150	ООО «Шаралов, Миронов и Лобанова»
152	ЗооМир

product_id	name	country	price	supplier_id
688	Лекарства антибиотикопоезд	Великобритания	2425.42	128
689	Игрушки интерактивные заплакать	Франция	1879.32	84
690	Аксессуары ошейники мальчишка	Италия	1073.95	85
691	Гигиена шампунь единый	Италия	3676.18	89
692	Корм премиум столетие	Франция	3033.34	145
693	Лекарства антибиотикопоезд	Испания	1693.58	36
694	Гигиена расчески покидать	Италия	4415.35	117
695	Аксессуары поводки заявление	Италия	3995.66	71
696	Аксессуары лежанки бегать	Китай	846.21	37
697	Корм холистик угол	Китай	3465.62	4
698	Корм холистик горючий	США	4110.85	58
699	Корм премиум банда	Франция	97.78	39
711	Наполнитель для лотка	Россия	800.50	30
712	Корм для кошек Premium	Норвегия	1200.00	152

Рис. 18: Таблицы `supplier` и `product` с новыми записями

6 Транзакции

Необходимо создать двух новых пользователей. Также нужно выбрать уровень изоляции и аномалию базы данных. Необходимо написать сценарий выполнения транзакций для двух созданных пользователей и продемонстрировать наличие или отсутствие выбранной аномалии на выбранном уровне изоляции.

Постановка задачи

1. Создать двух пользователей.
2. Выбрать уровень изоляции.
3. Выбрать аномалию.
4. Написать сценарий для каждого из пользователей о выполняемых ими транзакциях.
5. Продемонстрировать наличие или отсутствие выбранной аномалии на выбранном уровне изоляции.

6.1 Сценарий выполнения транзакций

Для выполнения работы были выбраны уровень изоляции READ UNCOMMITTED и аномалия «грязное чтение». Сценарий выполнения транзакций и результаты всех действий пользователей представлены в таблице 6.

Таблица 6: Сценарий выполнения транзакций

№	Действия транзакции user1	Действия транзакции user2
1	START TRANSACTION; UPDATE product SET price = price * 1.1 WHERE product_id = 1; user1 начинает транзакцию и изменяет цену товара на 1002.58.	—
2	—	START TRANSACTION; SELECT price FROM product WHERE product_id = 1; user2 начинает транзакцию и просматривает цену товара: 1002.58.

№	user1	user2
3	ROLLBACK; user1 откатывает свою транзакцию.	—
4	—	SELECT price FROM product WHERE product_id=1; COMMIT; user2 снова просматривает цену товара и получает значение 911.44.

6.2 Результаты выполнения транзакций

На уровне изоляции `READ_UNCOMMITTED` проявляется аномалия «грязного чтения», потому что на шагах 2 и 4 транзакции `user2` для одного и того же запроса получает разные значения цены товара: 1002.58 на шаге 2, 911.44 на шаге 4. Это происходит потому, что вносимые пользователем `user1` на шаге 1 изменения в рамках еще не завершенной транзакции видны и пользователю `user2` на шаге 2.

Результаты выполненных шагов приведены на рис. 5-7.

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE product SET price = price * 1.1 WHERE product_id = 1;
Query OK, 1 row affected, 1 warning (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 1

mysql> SELECT price FROM product WHERE product_id = 1;
+-----+
| price |
+-----+
| 1002.58 |
+-----+
1 row in set (0.00 sec)
```

Рис. 5: Начало транзакции пользователя `user1`

```
mysql> SELECT price FROM product WHERE product_id = 1;
+-----+
| price |
+-----+
| 1002.58 |
+-----+
1 row in set (0.00 sec)
```

Рис. 6: Начало транзакции пользователя `user2`

```
mysql> SELECT price FROM product WHERE product_id = 1;
+-----+
| price |
+-----+
| 1002.58 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT price FROM product WHERE product_id = 1;
+-----+
| price |
+-----+
| 911.44 |
+-----+
1 row in set (0.00 sec)
```

Рис. 7: Грязное чтение

Заключение

В ходе работы была спроектирована и реализована система управления данными для ветеринарной клиники на основе MySQL. Выполнение включало пять основных этапов: создание агрегирующего представления о покупках клиентов, разработку триггеров для автоматического учёта продаж, настройку доступа с разными правами пользователей, написание хранимой функции для форматирования ФИО и процедуры для добавления товаров и поставщиков, а также моделирование транзакций с демонстрацией аномалии «грязного чтения» на уровне изоляции READ UNCOMMITTED. В результате была создана функциональная основа информационной системы, которая обеспечивает целостность данных, автоматизацию ключевых операций и безопасный контроль доступа. Полученное решение демонстрирует практическую применимость изученных механизмов СУБД.

Список литературы

- [1] **Официальная документация по MySQL 8.0.** MySQL AB, 2024.
[\[Электронный ресурс\]](#) (дата обращения: 22.09.2015).
- [2] **Введение в базы данных, 8-е издание.** – М.: Издательский дом «Вильямс», 2005 – 1328 с.: – [\[Электронный ресурс\]](#) (дата обращения: 14.10.25).
- [3] **Takashi M., Azuma S.** – The Manga Guide to Databases. — 1st ed. – San Francisco: No Starch Press, 2009. – 224 с.: – [\[Электронный ресурс\]](#) (дата обращения: 03.11.25).

Приложение А. «Исходный текст функции»

```
1 DELIMITER //
```

```
2
```

```
3 CREATE FUNCTION get_surname_with_initials(  
4     second_name VARCHAR(50),  
5     first_name  VARCHAR(50),  
6     middle_name VARCHAR(50)  
7 )  
8 RETURNS VARCHAR(152)  
9 DETERMINISTIC  
10 BEGIN  
11     RETURN CONCAT(  
12         second_name, ' ',  
13         UPPER(SUBSTRING(first_name, 1, 1)), ' ',  
14         CASE  
15             WHEN middle_name IS NOT NULL AND middle_name != ''  
16             THEN CONCAT(UPPER(SUBSTRING(middle_name, 1, 1)), ' ')  
17             ELSE ''  
18         END  
19     );  
20 END //
```

```
21
```

```
22 DELIMITER ;
```

Листинг 17: Исходный код создания функции get_surname_with_initials

Приложение Б. «Исходный текст процедуры»

```
1 DELIMITER //
```

```
2
```

```
3 CREATE PROCEDURE add_product_with_supplier(  
4     IN supplier_name_in VARCHAR(50),  
5     IN product_name_in VARCHAR(50),  
6     IN country_in VARCHAR(15),  
7     IN price_in DECIMAL(10,2)  
8 )  
9 BEGIN  
10     DECLARE supplier_id_new INT;  
11     DECLARE product_id_existing INT;  
12     DECLARE supplier_count INT;  
13  
14     SELECT COUNT(*) INTO supplier_count  
15     FROM supplier  
16     WHERE name = supplier_name_in;  
17  
18     IF supplier_count = 0 THEN  
19         INSERT INTO supplier (name)  
20         VALUES (supplier_name_in);  
21  
22         SET supplier_id_new = LAST_INSERT_ID();  
23  
24         SELECT CONCAT('Добавлен новый поставщик: ',  
25             supplier_name_in, ' с ID: ', supplier_id_new) AS  
26             message;  
27     ELSE  
28         SELECT supplier_id INTO supplier_id_new  
29         FROM supplier  
30         WHERE name = supplier_name_in;  
31  
32         SELECT CONCAT('Поставщик уже существует. ID: ',  
33             supplier_id_new) AS message;  
34     END IF;  
35  
36     SELECT COUNT(*) INTO product_id_existing  
37     FROM product  
38     WHERE name = product_name_in  
39         AND country = country_in  
40         AND price = price_in  
41         AND supplier_id = supplier_id_new;  
42  
43     IF product_id_existing = 0 THEN  
44         INSERT INTO product (name, country, price, supplier_id)  
45         VALUES (product_name_in, country_in, price_in,  
46             supplier_id_new);  
47  
48         SELECT CONCAT('Добавлен новый товар: ', product_name_in,  
49             ' от поставщика ', supplier_name_in) AS message;
```

```

45     ELSE
46         SELECT CONCAT('Товар "', product_name_in, '" от поставщик
           а "', supplier_name_in, '" уже существует') AS message
           ;
47     END IF;
48
49 END //
50
51 DELIMITER ;

```

Листинг 18: Исходный код создания процедуры add_product_with_supplier